06 Functions

> https://youtu.be/FOD408a0EzU

JavaScript Functions: Complete Lecture Notes & Lab Guide

Setup Instructions

Before we begin, create a new folder for this lab and set up your files:

1. Create a folder named `js-functions-lab`
2. Inside it, create two files:
   - `index.html`
   - `script.js`

index.html:

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Functions Lab</title>
</head>
<body>
    <h1>JavaScript Functions Lab</h1>
    <div class="output">
        <p>Open your browser console (F12) to see the output!</p>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

3. Right-click on `index.html` and select "Open with Live Server"
4. Open the browser console (F12 or Right-click → Inspect → Console tab)

Part 1: Function Basics

What is a Function?

A function is a reusable block of code that performs a specific task. Think of it like a recipe—you write it once and use it many times.

Add to script.js:

```JS
console.log("=== PART 1: FUNCTION BASICS ===\n");

// Function Declaration
function greet() {
    console.log("Hello, World!");
}

// Calling the function
greet();
greet(); // We can call it multiple times!
```

Save and check the console. You should see "Hello, World!" printed twice.

Key Concept:

- **Function Declaration**: Define the function with the `function` keyword
- **Function Call**: Execute the function by writing its name followed by `()`

Part 2: Functions with Parameters

Parameters allow functions to accept input data. They make functions flexible and reusable.

Add this to script.js:

```JS
console.log("\n=== PART 2: PARAMETERS ===\n");

// Function with one parameter
function greetPerson(name) {
    console.log("Hello, " + name + "!");
}

greetPerson("Alice");
greetPerson("Bob");

// Function with multiple parameters
function introduce(name, age, city) {
    console.log(name + " is " + age + " years old and lives in " + city);
}

introduce("Charlie", 25, "New York");
introduce("Diana", 30, "London");
```

Check the console to see personalized messages.

Understanding Parameters:

- **Parameters** are variables listed in the function definition
- **Arguments** are the actual values passed when calling the function
- You can have as many parameters as needed

Let's try something fun:

```JS
// Function with default parameters (ES6 feature)
function orderCoffee(size = "Medium", drink = "Latte") {
    console.log("Ordering: " + size + " " + drink);
}

orderCoffee(); // Uses defaults
orderCoffee("Large"); // Uses Large, default for drink
orderCoffee("Small", "Cappuccino"); // Both specified
```

Part 3: The Return Statement

The `return` statement sends a value back from the function. This is how functions produce results!

Add this code:

```JS
console.log("\n=== PART 3: RETURN STATEMENT ===\n");

// Function that returns a value
function add(a, b) {
    return a + b;
}

let sum = add(5, 3);
console.log("5 + 3 = " + sum);

// Using the returned value directly
console.log("10 + 20 = " + add(10, 20));

// Function without return (returns undefined)
function sayHi() {
    console.log("Hi there!");
}

let result = sayHi();
console.log("Result of sayHi:", result); // undefined
```

Important Rules:

- `return` stops function execution immediately
- Only one value can be returned (but it can be an object or array)
- Functions without `return` automatically return `undefined`

More return examples:

```JS
// Return stops execution
function checkAge(age) {
    if (age < 18) {
        return "Too young";
    }
    return "Old enough";
    console.log("This never runs!"); // Code after return is unreachable
}

console.log(checkAge(15));
console.log(checkAge(25));

// Returning objects
function createPerson(name, age) {
    return {
        name: name,
        age: age,
        isAdult: age ≥ 18
    };
}

let person = createPerson("Eve", 22);
console.log(person);
```

Part 4: Function Expressions

Functions can be stored in variables! This is called a function expression.

Add this code:

```JS
console.log("\n═══ PART 4: FUNCTION EXPRESSIONS ═══\n");

// Regular function declaration
function declaredFunction() {
    console.log("I'm a declared function");
}

// Function expression (anonymous function stored in variable)
const expressedFunction = function() {
    console.log("I'm a function expression");
};

// Call both
declaredFunction();
expressedFunction();

// Function expression with parameters
const multiply = function(x, y) {
    return x * y;
};

console.log("4 × 7 =", multiply(4, 7));
```

Key Difference:

- **Function declarations** are hoisted (can be called before they're defined)
- **Function expressions** are not hoisted (must be defined before calling)

Part 5: Arrow Functions (Modern JavaScript)

Arrow functions are a shorter, modern way to write functions. They're very popular in modern JavaScript!

Add this code:

```JS
console.log("\n=== PART 5: ARROW FUNCTIONS ===\n");

// Traditional function
const traditionalFunc = function(name) {
    return "Hello, " + name;
};

// Arrow function (same thing, shorter!)
const arrowFunc = (name) => {
    return "Hello, " + name;
};

console.log(traditionalFunc("Frank"));
console.log(arrowFunc("Grace"));

// Even shorter: one-line arrow functions
const greetShort = name => "Hello, " + name;
console.log(greetShort("Henry"));

// Arrow function with multiple parameters
const addNumbers = (a, b) => a + b;
console.log("15 + 25 =", addNumbers(15, 25));

// Arrow function with no parameters
const getRandomNumber = () => Math.random();
console.log("Random number:", getRandomNumber());

// Arrow function with multiple lines
const calculateArea = (length, width) => {
    const area = length * width;
    console.log("Calculating area...");
    return area;
};

console.log("Area:", calculateArea(5, 10));
```

Arrow Function Rules:

1. **Basic syntax**: `(params) => { body }`
2. **Single parameter**: Parentheses optional `name => { body }`
3. **No parameters**: Empty parentheses required `() => { body }`
4. **Single expression**: Can omit `{}` and `return` → `(x, y) => x + y`
5. **Multiple lines**: Need `{}` and explicit `return`

Part 6: Const vs Let vs Var for Functions

Understanding how to properly declare functions is important.

Add this code:

```JS
console.log("\n=== PART 6: CONST vs LET for FUNCTIONS ===\n");

// Using const (recommended for functions)
const permanentFunc = () => {
    return "I cannot be reassigned";
};

console.log(permanentFunc());

// This would cause an error:
// permanentFunc = () => { return "New function"; }; // Error!

// Using let (allows reassignment)
let changeableFunc = () => {
    return "Original function";
};

console.log(changeableFunc());

changeableFunc = () => {
    return "I've been changed!";
};

console.log(changeableFunc());

// Best practice: Use const for functions
const bestPractice = () => {
    return "Use const for functions that won't be reassigned";
};

console.log(bestPractice());
```

Best Practice:

- **Use** `const` for function expressions and arrow functions (prevents accidental reassignment)
- Only use `let` if you specifically need to reassign the function
- Avoid `var` for modern JavaScript

Part 7: Practical Examples

Let's build some real-world functions!

Add this code:

JS

```
console.log("\n═══ PART 7: PRACTICAL EXAMPLES ═══\n");

// Example 1: Calculator
const calculator = {
    add: (a, b) => a + b,
    subtract: (a, b) => a - b,
    multiply: (a, b) => a * b,
    divide: (a, b) => b !== 0 ? a / b : "Cannot divide by zero"
};

console.log("Calculator:");
console.log("10 + 5 =", calculator.add(10, 5));
console.log("10 - 5 =", calculator.subtract(10, 5));
console.log("10 × 5 =", calculator.multiply(10, 5));
console.log("10 ÷ 5 =", calculator.divide(10, 5));

// Example 2: Temperature converter
const celsiusToFahrenheit = celsius => (celsius * 9/5) + 32;
const fahrenheitToCelsius = fahrenheit => (fahrenheit - 32) * 5/9;

console.log("\nTemperature Conversion:");
console.log("25°C =", celsiusToFahrenheit(25).toFixed(1) + "°F");
console.log("77°F =", fahrenheitToCelsius(77).toFixed(1) + "°C");

// Example 3: String utilities
const capitalize = str => str.charAt(0).toUpperCase() + str.slice(1);
const reverseString = str => str.split('').reverse().join('');
const wordCount = str => str.trim().split(/\s+/).length;

console.log("\nString Utilities:");
console.log("Capitalize 'hello':", capitalize("hello"));
console.log("Reverse 'javascript':", reverseString("javascript"));
console.log("Word count 'I love JavaScript':", wordCount("I love JavaScript"));

// Example 4: Array helpers
const getMax = arr => Math.max( ... arr);
const getMin = arr => Math.min( ... arr);
const getAverage = arr => arr.reduce((sum, num) => sum + num, 0) / arr.length;

const numbers = [12, 45, 67, 23, 89, 34];
console.log("\nArray Helpers:");
console.log("Numbers:", numbers);
console.log("Maximum:", getMax(numbers));
```

```
console.log("Minimum:", getMin(numbers));
console.log("Average:", getAverage(numbers).toFixed(2));
```

Part 8: Callback Functions

Functions can be passed as arguments to other functions! These are called callbacks.

Add this code:

```JS
console.log("\n═══ PART 8: CALLBACK FUNCTIONS ═══\n");

// Function that takes another function as parameter
const processArray = (arr, callback) => {
    const results = [];
    for (let item of arr) {
        results.push(callback(item));
    }
    return results;
};

const numbers2 = [1, 2, 3, 4, 5];

// Double each number
const doubled = processArray(numbers2, num => num * 2);
console.log("Original:", numbers2);
console.log("Doubled:", doubled);

// Square each number
const squared = processArray(numbers2, num => num ** 2);
console.log("Squared:", squared);

// Real-world example: setTimeout
console.log("\nExecuting delayed function...");
setTimeout(() => {
    console.log("This message appears after 2 seconds!");
}, 2000);

// Array methods with callbacks
const fruits = ["apple", "banana", "cherry"];

console.log("\nArray Methods with Callbacks:");
fruits.forEach(fruit => console.log("I like", fruit));

const upperFruits = fruits.map(fruit => fruit.toUpperCase());
console.log("Uppercase fruits:", upperFruits);

const longFruits = fruits.filter(fruit => fruit.length > 5);
console.log("Fruits with > 5 letters:", longFruits);
```

Part 9: Exercise Time!

Now it's your turn! Complete these exercises below the existing code.

Add this section:

JS

```
console.log("\n=== PART 9: EXERCISES ===\n");

// Exercise 1: Create a function that calculates the area of a circle
// Formula: π × radius²
const circleArea = radius => {
    // Your code here
    return Math.PI * radius ** 2;
};

console.log("Area of circle (radius 5):", circleArea(5).toFixed(2));

// Exercise 2: Create a function that checks if a number is even
const isEven = num => {
    // Your code here
    return num % 2 === 0;
};

console.log("Is 4 even?", isEven(4));
console.log("Is 7 even?", isEven(7));

// Exercise 3: Create a function that finds the largest of three numbers
const findLargest = (a, b, c) => {
    // Your code here
    return Math.max(a, b, c);
};

console.log("Largest of 10, 25, 15:", findLargest(10, 25, 15));

// Exercise 4: Create a function that generates a greeting based on time
const timeGreeting = hours => {
    // Your code here
    if (hours < 12) return "Good morning";
    if (hours < 18) return "Good afternoon";
    return "Good evening";
};

console.log("Greeting at 9am:", timeGreeting(9));
console.log("Greeting at 2pm:", timeGreeting(14));
console.log("Greeting at 8pm:", timeGreeting(20));

// Exercise 5: Create a function that counts vowels in a string
const countVowels = str => {
    // Your code here
    const vowels = 'aeiouAEIOU';
    let count = 0;
```

```
    for (let char of str) {
        if (vowels.includes(char)) count++;
    }
    return count;
};

console.log("Vowels in 'JavaScript':", countVowels("JavaScript"));
console.log("Vowels in 'Hello World':", countVowels("Hello World"));
```

Part 10: Summary & Best Practices

Add this final section:

JS

```
console.log("\n═══ SUMMARY ═══\n");

console.log(`
FUNCTION TYPES:
1. Function Declaration: function name() {}
2. Function Expression: const name = function() {}
3. Arrow Function: const name = () => {}

WHEN TO USE WHAT:
- Arrow functions: Short, simple operations
- Regular functions: Complex logic, methods
- Use const: For functions that won't be reassigned

KEY CONCEPTS:
✓ Parameters: Input to functions
✓ Return: Output from functions
✓ Callbacks: Functions as arguments
✓ Default parameters: Fallback values
✓ Arrow syntax: Modern, concise

BEST PRACTICES:
• Use descriptive function names (calculateTotal, not calc)
• Keep functions small and focused (do one thing well)
• Use const for function declarations
• Prefer arrow functions for simple operations
• Always return a value when expected
• Use default parameters when appropriate
`);

console.log("🎉 Congratulations! You've completed the Functions Lab!");
```

Quick Reference Card

```JS
// Function Declaration
function myFunc(param) {
    return param * 2;
}

// Function Expression
const myFunc = function(param) {
    return param * 2;
};

// Arrow Function (long form)
const myFunc = (param) => {
    return param * 2;
};

// Arrow Function (short form)
const myFunc = param => param * 2;

// Multiple Parameters
const add = (a, b) => a + b;

// No Parameters
const greet = () => "Hello!";

// Default Parameters
const greet = (name = "Guest") => `Hello ${name}`;

// Callback Example
numbers.map(num => num * 2);
```

Next Steps

1. Experiment with modifying the functions
2. Create your own functions for different tasks
3. Try combining multiple functions together
4. Look into more advanced topics: closures, IIFE, async functions

Happy Coding! 🚀